ELEC 475: Lab 4 — In-Lab

Semantic Segmentation & Knowledge Distillation

Naser Al-Obeidat and Michael Greenspan

5 Nov. 2026



Table of Contents

Step 1: In-Lab Files 2
Step 2: Student Model 3
Step 3: Response-Based Knowledge Distillation 4
Step 4: Feature-Based Knowledge Distillation 5
Step 5: Kick-Starting Training 6
Step 6: Completion 7

Step 1: In-Lab Files

> *You may use any large language model (e.g., ChatGPT, Claude) for this assignment. When prompting, be explicit and precise about what you want. Always test and fact-check every piece of code the model generates — do not accept code outputs at face value. Verify imports, function names, arguments, and tensor shapes, etc. - and of course make sure that it runs correctly!*

Download `lab4_inlab.zip` from the course Drive and extract it into your Lab 4 folder, alongside your Pre-Lab code. It contains these files:

- `train.py`: complete. Trains the student, with or without knowledge distillation.
- `test_kd.py`: complete. Checks your loss functions against reference answers.
- `evaluate.py`: complete. Compares the teacher and your trained students (to be used in the Post-Lab).
- `models/student.py`: complete. The student network.
- `utils/teacher.py`: complete. Loads the frozen FCN-ResNet50 teacher and returns its outputs and features.
- `utils/dataset.py`, `utils/metrics.py`, `utils/profiler.py`, `utils/visualization.py`: complete.
- `utils/kd.py`: to be completed in Steps 3 and 4.

Activate your `.venv`, then confirm that your Pre-Lab dataset is in place:

```
# On macOS / Linux / WSL Bash (Course Standard):
source .venv/bin/activate

# On Windows PowerShell:
.\.venv\Scripts\Activate.ps1

# Confirm your Pre-Lab dataset is in place:
python download_voc.py
```

It must print `train: 1464 val: 1449`. Make sure the `download_and_extract_archive` line is commented out, so the dataset is not downloaded again.

Step 2: Student Model

The student is a compact network built on a pretrained MobileNetV3-Small backbone, followed by an ASPP module and a light decoder. It will learn from the teacher, the pretrained FCN-ResNet50. Run the following line to inspect the student:

```
python -m models.student
```

It should report **3,265,173** parameters, about 9% of the teacher's 35,322,218. Its `forward` method returns a dictionary with the 21-class scores (`"out"`) and three backbone features, which you will use in Step 4:

```
out   (1, 21, 320, 320)
low   (1, 24, 40, 40)
mid   (1, 40, 20, 20)
high  (1, 576, 10, 10)
```

In this lab, you will write the two knowledge distillation losses and start training with each of them. The full training runs and the comparison are part of the Post-Lab.

Step 3: Response-Based Knowledge Distillation

In response-based distillation, the student learns from the teacher's output: its per-pixel class probabilities. The loss is:

$$\mathcal{L}(x;W) = \alpha * \mathcal{H}(y, \sigma(z_s; T=1)) + \beta * \mathcal{H}(\sigma(z_t; T=\tau), \sigma(z_s, T=\tau))$$

where z_s and z_t are the student and teacher logits, y is the ground truth, H is the cross-entropy, and σ(z; T) = softmax(z / T). The first term is the usual loss with the ground truth. The second term makes the student match the teacher's probabilities, softened by a temperature τ > 1. Softening shows which wrong classes the teacher finds similar (e.g. cat and dog), which a one-hot label cannot.

Complete `response_kd_loss` in `utils/kd.py`:

Function	Requirements
`response_kd_loss( z_s, z_t, y, alpha, beta, tau, ignore_index)`	`z_s` and `z_t` are (N, 21, H, W) logits and `y` is an (N, H, W) label tensor. Return the loss above as a scalar tensor.

1. Compute the first term with `F.cross_entropy(z_s, y, ignore_index=255)`.
2. Compute the teacher probabilities `F.softmax(z_t / tau, dim=1)` and the student log-probabilities `F.log_softmax(z_s / tau, dim=1)`.
3. For each pixel, the second term is −Σ (teacher probability × student log-probability) over the 21 classes. Average it over the pixels where `y` is not 255.
4. Return `alpha * first + beta * second`.

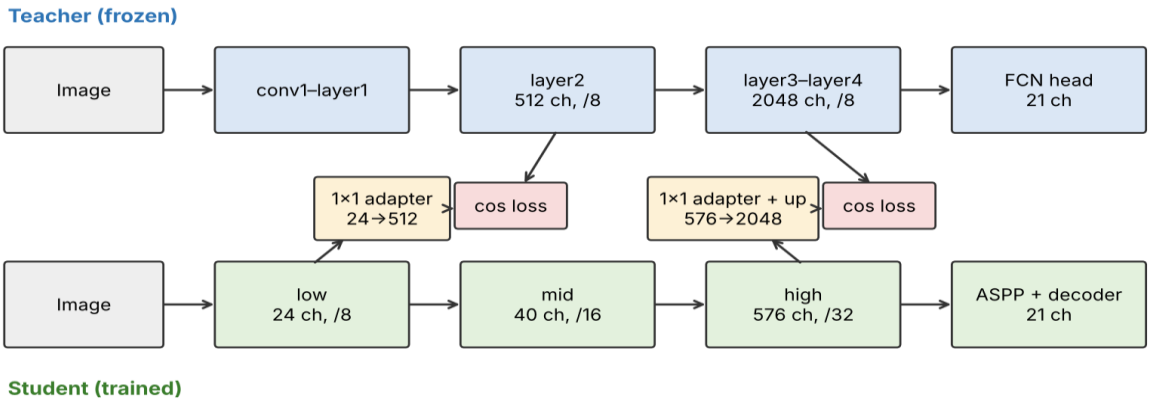
Check your function:

```
python test_kd.py
```

The three `response` checks must print PASS. The `feature` checks will print TODO until Step 4.

Step 4: Feature-Based Knowledge Distillation

In feature-based distillation, the student learns from the teacher's intermediate feature maps rather than its output. Two of the student's features are compared with two of the teacher's:



Student feature	Teacher feature	Adapter
low: 24 ch, stride 8	layer2: 512 ch, stride 8	1 × 1 conv, 24 → 512
high: 576 ch, stride 32	layer4: 2048 ch, stride 8	1 × 1 conv, 576 → 2048, then upsample

The features have different channel counts, so a 1 × 1 convolution (an adapter) maps each student feature to the teacher's channel count. The adapters are trained along with the student and discarded afterwards, so they are not counted in the student's parameters.

Each pair is compared with the cosine loss: at every pixel, the student and teacher feature vectors (across channels) should point in the same direction:

$$L_{\text{feat}} = \text{mean over pixels of } [\, 1 - \cos(\, f_s, f_t \,) \,]$$

The total loss is the cross-entropy with the ground truth plus γ times the sum of the two feature losses, with γ = 1. Complete `feature_kd_loss` in `utils/kd.py`:

Function	Requirements
`feature_kd_loss(f_s, f_t)`	`f_s` is an (N, C, h, w) adapted student feature and `f_t` is an (N, C, H, W) teacher feature. Resize `f_s` to (H, W) with bilinear interpolation, then return the mean of `1 - F.cosine_similarity(f_s, f_t, dim=1)`.

Run `python test_kd.py` again. All seven checks must print PASS:

```
7/7 checks passed
```

Step 5: Kick-Starting Training

With both losses complete, start training the student with each method. The `--iters 20` option stops each run after 20 iterations, which is enough to check that training works:

```
python train.py --kd none --iters 20
python train.py --kd response --iters 20
python train.py --kd feature --iters 20
```

Each run prints the loss every 5 iterations and ends with a summary:

```
iter   5/20  loss x.xxx
...
iter  20/20  loss x.xxx
Kick-start finished: average loss x.xxx (first 5) -> x.xxx (last 5)
Saved plots/loss_<kd>.png
```

In all three runs, the average loss over the last 5 iterations should be lower than over the first 5. If a loss is NaN or increases, check your loss functions before moving on.

The `response` and `feature` runs also run the teacher, in inference mode with frozen weights, so they are slower than the `none` run. If you run out of GPU memory, lower the batch size, e.g. `-b 8`.

Step 6: Completion

To have your in-lab work marked as complete, you must show a TA the following:

- The output of `python test_kd.py`, with 7/7 checks passed.
- The output of the three kick-start runs, and `plots/loss_none.png`, `plots/loss_response.png` and `plots/loss_feature.png`

Be ready to explain to the TA:

1. What each term of the response-based loss does, and why it uses a temperature $\tau > 1$.
2. Why feature-based distillation needs the adapters, and why they are not counted in the student's parameters.
3. What the cosine loss measures, and why it equals 0 for identical features and 2 for opposite ones.

The marking rubric is as follows:

Item	**Mark**
Response-based loss: test_kd.py checks pass	1
Feature-based loss: test_kd.py checks pass	1
Kick-start runs: loss decreases in all three	0.5
Explanation	0.5
Total	**3**

The remaining lab work (the full training runs, the comparison with `evaluate.py` and the report) will be part of the Post-Lab.